МИНИСТЕРСТВО науки и высшего ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГБОУ ВО «Московский авиационный институт (национальный исследовательский университет)»
Институт №4 «Системы и средства связи»
Кафедра системного программирования и информационных технологий

ОТЧЁТ

по лабораторным работам №1 и №2
«Взаимодействие с web-клиентом. Разработка мобильного клиента»
Вариант №7: сервер статистики аналитики сайта

Выполнил: студент группы ИУ7-XX
Плоцкий Б. А.
Проверил: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1. Задание....4
1.1 Состав отчёта....4
1.2 Требования лабораторной работы №1 и как они выполнены....5
1.3 Требования лабораторной работы №2 и как они выполнены....7
2. Статус выполнения работ....7
3. Выбор технологий и обоснование....9
4. Структура проекта....10
5. Работа №1. Сервер статистики и web-интерфейс....11
5.1 Модель данных....11
5.2 Сервер, хранение и найденные ошибки....12
5.3 HTTP API....13
5.4 Асинхронный сбор данных через JS....14
5.5 Формулы метрик....16
5.6 Web-интерфейс и обработка нажатий кнопок....16
5.7 Адаптивная вёрстка через @media....17
5.8 Визуализация результатов....17
5.9 Жизненный цикл сессии....21
6. Работа №2. Мобильный клиент под ОС Аврора....22
6.1 WebSocket-канал сервера....22
6.2 Клиент StatClient на C++....23
6.3 QML-интерфейс приложения....25
6.4 Сборка приложения в Aurora SDK....26
6.5 Установка и запуск на эмуляторе Aurora OS....27
7. Архитектура обеих работ....29
8. Тестирование....30
9. Демонстрация работы....31
9.1 Запуск в локальной сети....31
9.2 Сводка за демонстрационный прогон....31
9.3 Скриншоты: браузер десктопной ОС....32

9.4 Скриншоты: мобильный браузер (ОС Аврора).....37
9.5 Скриншот приложения на эмуляторе Аврора.....43
10. Проверка требований варианта №7.....44
11. Выводы.....44
12. Как воспроизвести материалы отчёта.....46
13. Полезные команды.....47

1. Задание

1.1 Состав отчёта

Первая лабораторная работа курса включает сразу два задания из файла заданий — №1 и №2. Они выполняются как один проект: сервер с web-интерфейсом и мобильное приложение под ОС Аврора, которое общается с этим сервером по web-socket-каналу. Вариант №7 в обоих заданиях одинаковый, поэтому сервер один, а не два.

Лабораторная работа №1. Взаимодействие с web-клиентом. Необходимо разработать серверное программное обеспечение на любом языке программирования, реализующее логику работы в соответствии с вариантом. Данное серверное обеспечение должно взаимодействовать с пользователем посредством браузера, иметь web-интерфейс. Для реализации web-интерфейса необходимо создать файлы html, css и js. Файлы js должны обеспечивать функции управления пользовательскими действиями, обрабатывать нажатия на кнопки web-формы. Кроме того, необходимо добавить асинхронный вызов js (async, promise и т. д.). В css файле обеспечить адаптивную верстку под различные разрешения экрана с использованием @media. Реализованное программное обеспечение запускается в локальной сети. Демонстрируется работа приложения в браузере десктопной ОС (ПК) и в браузере мобильного телефона под управлением ОС Аврора.

Лабораторная работа №2. Разработка мобильного клиента. Необходимо разработать приложение на Qt (возможны варианты с flutter) для запуска на мобильном устройстве под управлением ОС Аврора. Данное приложение должно взаимодействовать с сервером посредством передачи данных через сокеты (web-сокеты). Запустить сервер на удаленном устройстве (ПК), обеспечить подключение мобильного устройства к серверу в локальной сети и взаимодействие приложения с сервером.

Вариант №7: Сервер статистики аналитики сайта. Простое решение для сбора статистической информации о посетителях сайта: количество просмотров страниц, уникальные посетители, длительность сессий. Данные собираются асинхронно через JS и сохраняются на сервере с визуализацией результатов в виде графиков или диаграмм.

Таблица 1 — Состав отчёта

Работа	**Содержание**	**Код и артефакт**
№1	Сервер, web-интерфейс (html, css, js), адаптивная вёрстка, асинхронный сбор статистики	`stat-server/`: Node.js сервер, 5 страниц, панель статистики
№2	Приложение на Qt/QML для ОС Аврора, обмен с сервером по web-socket	`aurora-app/`: пакет RPM для Aurora OS 5.1.1.60
общая часть	WebSocket-канал `/ws` на сервере работы №1	`websocket.js`, `realtime.js`, 9 тестов канала

1.2 Требования лабораторной работы №1 и как они выполнены

Таблица 2 — Требования лабораторной работы №1 и как они выполнены

Требование задания	**Как выполнено**	**Файлы**
Серверное ПО на любом языке	HTTP-сервер на Node.js без внешних зависимостей (`http`, `fs`, `path`, `crypto`, `os`)	`server/server.js`, `server/storage.js`, `server/analytics.js`
Взаимодействие через браузер, web-интерфейс	5 страниц сайта + панель статистики (дашборд)	`public/*.html`
Файлы html	`index.html`, `catalog.html`, `about.html`, `contact.html`, `dashboard.html`	`public/`
Файлы css	`style.css` (сайт), `dashboard.css` (панель статистики)	`public/css/`
Файлы js	`api.js`, `tracker.js`, `charts.js`, `dashboard.js`, `app.js`	`public/js/`

Требование задания	Как выполнено	Файлы
JS управляет действиями пользователя, обрабатывает нажатия кнопок формы	Обработчики `submit`/`click`/`change`: фильтр каталога, форма обратной связи, форма отчёта, кнопки «Применить», «Обновить», «Демо-данные», «Экспорт CSV», «Сбросить», мобильное меню, «Показать ещё»	`public/js/app.js`, `public/js/dashboard.js`
Асинхронный вызов JS (`async`, `promise`)	`async/await` + `fetch` + `AbortController` + `Promise.all` + `navigator.sendBeacon`; очередь неотправленных событий, автообновление по таймеру	`public/js/api.js`, `public/js/tracker.js`, `public/js/dashboard.js`
Адаптивная вёрстка через `@media`	Брейкпоинты 992 px, 768 px, 480 px, плюс `prefers-color-scheme: dark` и `@media print`	`public/css/style.css`, `public/css/dashboard.css`
Запуск в локальной сети	Сервер слушает `0.0.0.0:3000` и печатает все IPv4-адреса машины для доступа с телефона	`server/config.js`, `server/server.js`
Демонстрация на ПК и на Аврора	Снимки десктопного браузера и мобильного браузера с User-Agent Аврора	раздел 9
Метрика: количество просмотров страниц	`pageViews` по событиям `pageview`	`server/analytics.js`
Метрика: уникальные посетители	`visitorId` в `localStorage`, учёт новых и вернувшихся посетителей	`public/js/tracker.js`, `server/storage.js`
Метрика: длительность сессий	Сессия = цепочка событий посетителя; закрытие по `session_end` или таймауту 1800 с	`server/storage.js`, `public/js/tracker.js`
Сбор данных асинхронно через JS	Фоновые POST-запросы `/api/events` при загрузке страницы, кликах, heartbeat каждые 30 с	`public/js/tracker.js`
Сохранение на сервере	Файл `data/stats.json`, асинхронная запись с debounce 400 мс	`server/storage.js`
Визуализация графиками или диаграммами	4 canvas-графика на дашборде + 6 графиков в отчёте	`public/js/charts.js`, `report/images/`

1.3 Требования лабораторной работы №2 и как они выполнены

Таблица 3 — Требования лабораторной работы №2 и как они выполнены

Требование задания	Как выполнено	Файлы
Приложение на Qt для мобильного устройства под ОС Аврора	`CONFIG += auroraapp`, `QT += websockets`, Qt 5.6.3 + Sailfish Silica, QML-страницы и C++-модель	`ru.mai.aurorastat.pro`, `src/`, `qml/`
Взаимодействие с сервером через web-сокеты	Класс `StatClient` на `QWebSocket`: подписка, приём снимка статистики, отправка своих событий	`src/statclient.cpp`, `src/statclient.h`
Сервер запущен на удалённом устройстве (ПК)	Сервер работы №1 слушает `0.0.0.0:3000` на ПК; приложение подключается к `ws://<адрес>:3000/ws`	`server/server.js`, `server/websocket.js`
Подключение мобильного устройства в локальной сети	Проверено на эмуляторе Aurora OS 5.1.1.60: рукопожатие `101 Switching Protocols`, сервер видит установленное соединение	раздел 6.5
Взаимодействие приложения с сервером	Двусторонний обмен: `subscribe`, `track` от клиента; `hello`, `stats`, `update`, `tracked`, `ping` от сервера	`server/realtime.js`, `src/statclient.cpp`
Отображение данных варианта №7 на мобильном устройстве	KPI, линейный график просмотров, кольцевая диаграмма устройств, столбцы страниц, таблица сессий	`qml/pages/`, `qml/components/`

2. Статус выполнения работ

Таблица 4 — Статус выполнения работ

Работа	Компонент	Статус	Подтверждение
№1	HTTP-сервер, API, хранение статистики	выполнено	`npm test` — 33 проверки из 33

Работа	Компонент	Статус	Подтверждение
№1	Web-интерфейс: 5 страниц, формы, кнопки, адаптивная вёрстка	выполнено	снимки разделов 9.3 и 9.4
№1	Графики на canvas и графики отчёта	выполнено	`charts.js`, `report/images/`
№1	Канал `/ws`	выполнено	`npm run test:ws` — 9 проверок из 9
№1	Smoke-тест интерфейса (16 проверок)	выполнено	`npm run test:ui` — 16 проверок из 16
№2	Код приложения (C++ + QML), 1749 строк	выполнено	`aurora-app/`
№2	Сборка в Aurora SDK 5.1.1.60 (цель x86_64)	выполнено	`sfdk build` → RPM в `RPMS/`
№2	Установка на устройство (эмулятор Aurora OS)	выполнено	`pkcon install-local`, `rpm -q`
№2	Запуск приложения и связь с сервером	выполнено	снимок 9.5, рукопожатие `101`
№2	Сборка под aarch64 (для телефона)	отложено	устройство — эмулятор x86_64

Таблица 5 — Чем подтверждён статус (команды и вывод)

Проверка	Команда	Результат
API сервера	`npm test`	`Итого: 33 из 33 проверок пройдено.`
Канал `/ws`	`npm run test:ws`	`Итого: 9 из 9 проверок пройдено.`
Сборка приложения	`sfdk build` (цель x86_64)	`Wrote: …/aurora-app/RPMS/…x86_64.rpm`
Установка на эмулятор	`sudo pkcon install-local -y /tmp/…rpm`	`Installed ru.mai.aurorastat`
Канал с устройства	`curl` с заголовками upgrade из гостя к `10.0.2.2:3000/ws`	`HTTP/1.1 101 Switching Protocols` + сообщение `hello`
Соединение видно серверу	`ss -tn state established '( sport = :3000 )'`	соединение от адреса эмулятора

3. Выбор технологий и обоснование

1. **Node.js, только стандартные модули.** Задание требует «любой язык» и запуск в локальной сети. Нулевые зависимости означают, что сервер запускается одной командой `node server/server.js` без установки пакетов и без интернета — важно для демонстрации на защите.
2. **Хранение в JSON-файле.** Аналитике не нужны транзакции и связи; файл `data/stats.json` читаемый, его можно показать руками, загрузка и сохранение — асинхронные операции `fs/promises`.
3. **Графики на canvas собственным кодом.** Внешние библиотеки (Chart.js) требуют CDN или сборки; на телефоне с Аврора сеть может быть ограничена, а canvas есть в браузере Авроры.
4. **WebSocket без npm-пакетов.** Канал `/ws` реализован вручную по RFC 6455 (`server/websocket.js`), чтобы работа №2 не добавила зависимостей в работу №1.
5. **Qt 5 + QML (Sailfish Silica) в приложении.** Это штатный стек Авроры: `CONFIG += auroraapp` и `QT += websockets` дают сборку через `sfdk build` и пакет RPM, который ставится на устройство пакетным менеджером.
6. **Логика клиента на C++, интерфейс на QML.** `StatClient` подписан через `Q_PROPERTY`, поэтому QML читает статистику как свойства и реагирует на сигналы; рисование графиков остаётся декларативным (`Canvas` в QML).
7. **Консервативный JavaScript в трекере.** Без optional chaining, nullish-оператора и стрелочных функций в критичном коде: браузер Авроры (Gecko/Sailfish) старше десктопных.
8. **Графики отчёта — Python/matplotlib.** Отдельный скрипт строит те же графики из того же файла `stats.json`, что даёт бумажную версию визуализации.

4. Структура проекта

```
lab01/
├── stat-server/                работа №1: сервер и web-интерфейс
│   ├── server/
│   │   ├── server.js           HTTP-сервер, маршруты /api/*, раздача
статики
│   │   ├── websocket.js        рукопожатие и кадры WebSocket (RFC
6455)
│   │   ├── realtime.js         подписки, рассылка stats/update по
каналу /ws
│   │   ├── storage.js          модель данных, запись на диск, сессии,
посетители
│   │   ├── analytics.js        расчёт метрик и агрегатов для графиков
│   │   ├── seed.js             генератор демонстрационных данных
│   │   ├── seed-cli.js         запуск генератора из командной строки
│   │   └── config.js           параметры (порт, таймауты, пути)
│   ├── public/                 web-интерфейс
│   │   ├── index.html          главная страница
│   │   ├── catalog.html        каталог с формой фильтра
│   │   ├── about.html          описание сервера статистики
│   │   ├── contact.html        форма обратной связи + таблица отзывов
│   │   ├── dashboard.html      панель статистики (графики, KPI,
таблицы)
│   │   ├── css/style.css       стили сайта + @media
│   │   ├── css/dashboard.css   стили панели + @media
│   │   ├── js/api.js           async-обёртка над fetch
│   │   ├── js/tracker.js       асинхронный сбор статистики с клиента
│   │   ├── js/charts.js        графики на canvas
│   │   ├── js/dashboard.js     форма отчёта, KPI, графики, таблицы
│   │   └── js/app.js           меню, живой счётчик, фильтр каталога,
отзывы
│   ├── data/stats.json         файл статистики (создаётся при запуске)
│   ├── test/                   тесты API, WebSocket, интерфейса
│   ├── docs/                   документация (10 файлов)
│   └── README.md
├── aurora-app/                 работа №2: приложение под ОС Аврора
│   ├── ru.mai.aurorastat.pro   qmake: CONFIG += auroraapp, QT +=
websockets
│   ├── src/main.cpp            Aurora::Application, contextProperty
StatClient
│   ├── src/statclient.h/.cpp   клиент WebSocket-канала (C++)
│   ├── qml/aurorastat.qml      корневой документ приложения
│   ├── qml/pages/              MainPage, DashboardPage, SessionsPage,
```

```
│   │                              SettingsPage, AboutPage
│   ├── qml/components/            StatCard, LineChart, DeviceDonut,
BarList
│   ├── qml/cover/CoverPage.qml    карточка приложения
│   ├── icons/                     86x86, 108x108, 128x128, 172x172
│   ├── translations/              ru.mai.aurorastat.ts, …-ru.ts
│   ├── rpm/ru.mai.aurorastat.spec   сборка пакета
│   └── ru.mai.aurorastat.desktop    запись меню приложений
└── report/                        материалы этого отчёта: схемы, графики,
снимки
```

Таблица 6 — Объём кода

Часть	**Строк**
сервер работы №1 (`server/*.js`)	1833
клиентский JS работы №1 (`public/js/`)	1375
HTML-страницы	543
CSS	494
тесты работы №1	693
приложение Авроры: C++ (`src/`)	570
приложение Авроры: QML (`qml/`)	1179
итого	**6687**

5. Работа №1. Сервер статистики и web-интерфейс

5.1 Модель данных

Три сущности: **посетитель**, **сессия**, **событие**. Посетитель идентифицируется UUID, который клиент хранит в `localStorage` (поэтому посетитель «уникальный» и может возвращаться). Сессия живёт в `sessionStorage` и на сервере; событие — это один запрос с клиента.



Рисунок 1 — Модель данных статистики

Таблица 7 — Модель данных

Сущность	Ключевые поля	Назначение
`visitor`	`id`, `firstSeen`, `lastSeen`, `visits`, `pageViews`, `ua`, `device`, `lang`, `screen`, `tz`	уникальные посетители, новые и вернувшиеся
`session`	`id`, `visitorId`, `start`, `lastEvent`, `end`, `closed`, `durationSec`, `pageViews`, `actions`, `entryPage`, `exitPage`, `referrer`, `device`	длительность сессий, глубина, отказ, вход и выход
`event`	`id`, `ts`, `type`, `visitorId`, `sessionId`, `page`, `label`, `referrer`, `device`, `ip`	просмотры страниц, нажатия кнопок, heartbeat
`feedback`	`ts`, `name`, `email`, `message`, `page`	демонстрация обработки web-формы

Типы событий: `pageview` (просмотр страницы), `action` (нажатие кнопки формы), `heartbeat` (страница открыта), `session_end` (страница закрыта).

5.2 Сервер, хранение и найденные ошибки

Сервер — `server/server.js` на встроенном `http`: маршрутизация `/api/*`, раздача статики из `public/`, лимит тела запроса, CSV-экспорт.

Хранение — `server/storage.js`: загрузка JSON при старте, накопление событий в памяти, асинхронная запись на диск с debounce 400 мс, ограничение размера (20000 событий, 5000 сессий).

Таблица 8 — Ошибки учёта, найденные при проверке демо-данных и исправленные

Проблема	**Симптом**	**Исправление**
Суффикс `#N` добавлялся идентификатору сессии даже без конфликта	35 реальных сессий считались как 85	суффикс создаётся только при коллизии: `while (state.sessions[sessionId]) { … }`
События добавлялись в массив не по времени	`firstEvent > lastEvent`, пустые окна таймлайна, неверное усечение	`insertEvent()` — бинарная вставка по `ts`, массив событий всегда отсортирован
Внешние источники не появлялись в статистике	все демо-сессии помечались «внутренними»	referrer-ы в `seed.js` стали взвешенными (`pickWeighted`)
«Внутренний переход» определялся только по Host заголовку	сайт, открытый по `localhost` и по `127.0.0.1`, считался внешним	клиент передаёт `origin` своей страницы, сервер сравнивает хост referrer-а с хостом страницы

5.3 HTTP API

Таблица 9 — HTTP API

Метод и маршрут	**Назначение**	**Ответ**
`GET /api/health`	проверка живости, версия, время работы	JSON
`GET /api/stats?range=&page=&limit=`	полный отчёт для дашборда	JSON с KPI, таймлайном, топ-страницами, устройствами, сессиями
`GET /api/pages`	список страниц для селектора формы отчёта	JSON
`GET /api/sessions?limit=`	последние сессии	JSON
`GET /api/feedback`	отзывы, отправленные через форму	JSON
`POST /api/events`	приём события трекера	`201` + краткая сводка

Метод и маршрут	Назначение	Ответ
	(`pageview`, `action`, `heartbeat`)	
`POST /api/session-end`	закрытие сессии, фиксация длительности	JSON с сессией
`POST /api/feedback`	сохранение web-формы	`201` + запись
`GET /api/export.csv?type=…`	выгрузка CSV (`;` разделитель, UTF-8 BOM для Excel)	`text/csv`
`POST /api/seed`	добавить демонстрационные данные	JSON
`POST /api/reset`	очистить статистику	JSON

5.4 Асинхронный сбор данных через JS

Клиентский трекер `public/js/tracker.js` отправляет события фоново, не мешая работе страницы. Путь события показан на схеме: от загрузки страницы до графика на дашборде.



Рисунок 2 — Путь события от браузера до графика

```
async function postJson(url, payload, useKeepalive) {
  var response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
    keepalive: Boolean(useKeepalive),
    cache: 'no-store',
    credentials: 'omit'
```

```
  });
  if (!response.ok) throw new Error('HTTP ' + response.status);
  return response.json().catch(function () { return {}; });
}

/** Отправляет событие; при неудаче кладёт его в очередь на повтор. */
async function sendEvent(type, extra) {
  if (!consent) return null;
  var payload = basePayload();
  payload.type = type;
  try {
    var result = await postJson(EVENT_URL, payload, false);
    flushQueue();
    return result;
  } catch (err) {
    if (queue.length < MAX_QUEUE) queue.push(payload);
    return null;
  }
}
```

Таблица 10 — Что делает трекер

Действие браузера	**Событие на сервер**	**Когда**
Открыл страницу	`pageview` (page, origin, referrer, ua, lang, screen, tz)	при загрузке
Нажал кнопку формы	`action` + `label` (например `catalog:filter`, `dashboard:apply`)	при клике
Страница открыта	`heartbeat`	каждые 30 с
Закрыл вкладку или ушёл	`session_end` через `fetch(keepalive)` или `navigator.sendBeacon`	`pagehide`, `visibilitychange`
Сеть недоступна	событие попадает в очередь и досылается через `flushQueue()`	при следующем успехе

5.5 Формулы метрик

Таблица 11 — Формулы метрик

Метрика	**Формула**
Просмотры страниц	сумма событий с `type == 'pageview'`
Уникальные посетители	число `visitors`, попавших в выбранный период
Новые и вернувшиеся	`firstSeen` внутри периода и вне периода
Длительность сессии	`(end − start) / 1000`, где `end = lastEvent` при закрытии
Средняя длительность сессии	сумма `durationSec` / число сессий
Глубина просмотра	сумма `pageViews` / число сессий
Показатель отказов	сессии с `pageViews == 1` / все сессии × 100 %
Активные сейчас	сессии с событием за последние 5 минут
Тип устройства	по User-Agent: `aurora` → `mobile` → `tablet` → `desktop`
Источник перехода	пусто → «прямой заход»; хост referrer равен хосту страницы (`origin`) → «внутри сайта»; иначе внешний хост

5.6 Web-интерфейс и обработка нажатий кнопок

Таблица 12 — Web-интерфейс и обработка нажатий кнопок

Страница	**Форма и кнопки**	**Обработчик**	**Что происходит**
`index.html`	живые счётчики, кнопка меню `#navToggle`	`app.js`	`GET /api/stats` через `async/await`, обновление счётчиков, отправка `action` при открытии меню
`catalog.html`	форма `#filterForm`: `#category`, `#maxPrice`, `#search`, кнопки «Применить» и «Показать ещё»	`app.js`	`submit` + `preventDefault`, фильтрация без перезагрузки страницы, `POST /api/events` с `label: catalog:filter`
`contact.html`	форма `#feedbackForm`:	`app.js`	клиентская валидация, `POST /`

Страница	Форма и кнопки	Обработчик	Что происходит
	имя, e-mail, тема, сообщение; кнопки `#sendBtn` и `#clearBtn`		`api/feedback`, вывод ошибок в `#formMessages`, обновление таблицы `#feedbackTable`
`dashboard.html`	форма отчёта `#reportForm`: период `#range`, страница `#page`, лимит `#limit`, автообновление `#autoRefresh`; кнопки «Применить», «Обновить», «Демо-данные», «Экспорт CSV», «Сбросить»	`dashboard.js`	`loadStats()` — `async`, параллельные запросы через `Promise.all`, перерисовка KPI, графиков и таблиц, `setInterval` на 5 с
все страницы	—	`tracker.js`	асинхронная отправка статистики

5.7 Адаптивная вёрстка через `@media`

Таблица 13 — Адаптивная вёрстка через @media

Правило	Что меняется
`@media (max-width: 992px)`	сетка карточек в 2 колонки, панель статистики в 1 колонку, canvas на всю ширину
`@media (max-width: 768px)`	горизонтальное меню заменяется кнопкой `#navToggle`, KPI-сетка в 2 колонки, таблицы прокручиваются горизонтально
`@media (max-width: 480px)`	одна колонка, формы на всю ширину, шрифт 15 px, кнопки формы отчёта друг под другом
`@media (prefers-color-scheme: dark)`	тёмная палитра (удобно на AMOLED-телефоне)
`@media print` (dashboard.css)	печать отчёта без кнопок и меню

5.8 Визуализация результатов

Дашборд рисует 4 графика на `<canvas>` собственным кодом (`charts.js`): линия по времени, горизонтальные бары по страницам,

кольцевая диаграмма устройств, гистограмма длительности сессий. Canvas пересчитывается при изменении размера окна с учётом `devicePixelRatio`, поэтому графики корректны и на ПК, и на телефоне.

Те же данные в отчёте визуализированы скриптом `report/src/python/make_charts.py` (matplotlib) напрямую из файла статистики.

Просмотры страниц по времени (корзины по 6 часов, 7 дней)

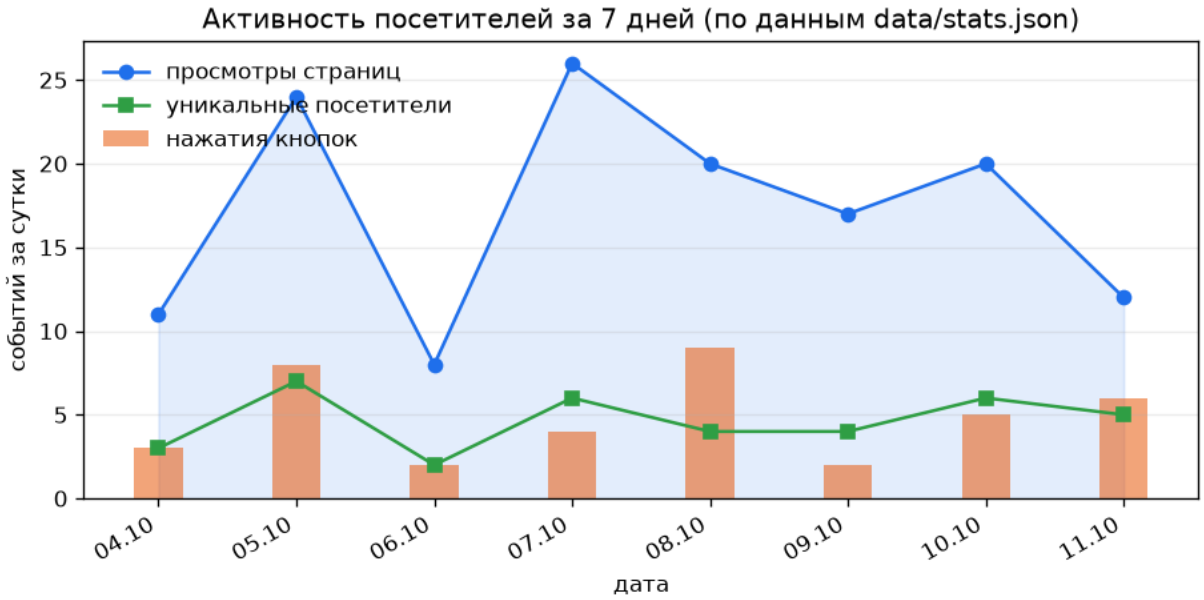


Рисунок 3 — Таймлайн просмотров

Популярные страницы

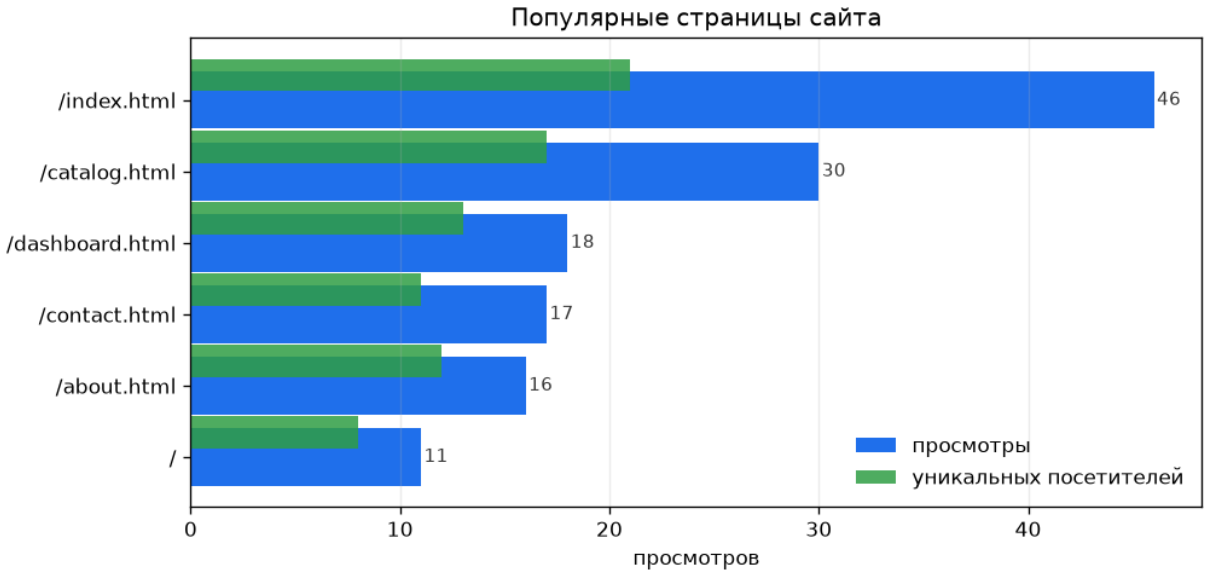


Рисунок 4 — Топ страниц

Устройства посетителей (диаграмма)



Рисунок 5 — Устройства посетителей

Распределение сессий по длительности

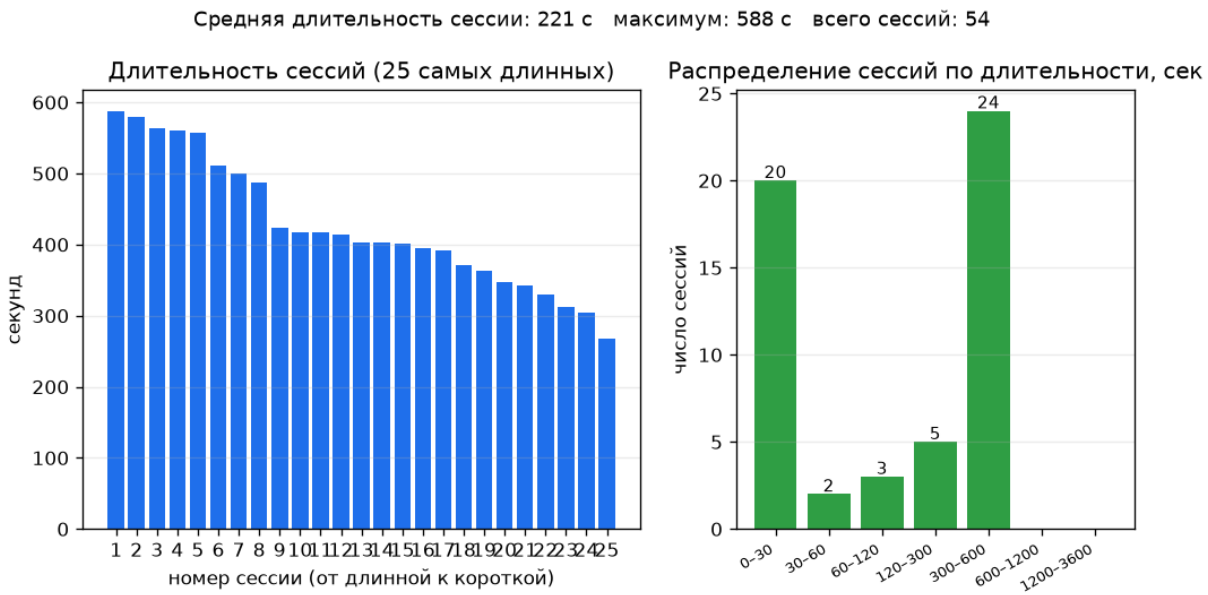


Рисунок 6 — Длительность сессий

Источники переходов



Рисунок 7 — Источники переходов

Сводка ключевых показателей



Рисунок 8 — Ключевые показатели

5.9 Жизненный цикл сессии

Событие того же `sessionId` продлевает сессию. Если между событиями прошло больше `SESSION_TIMEOUT_SEC` (1800 с) — старая сессия закрывается с `durationSec = lastEvent − start`, а новая начинается.

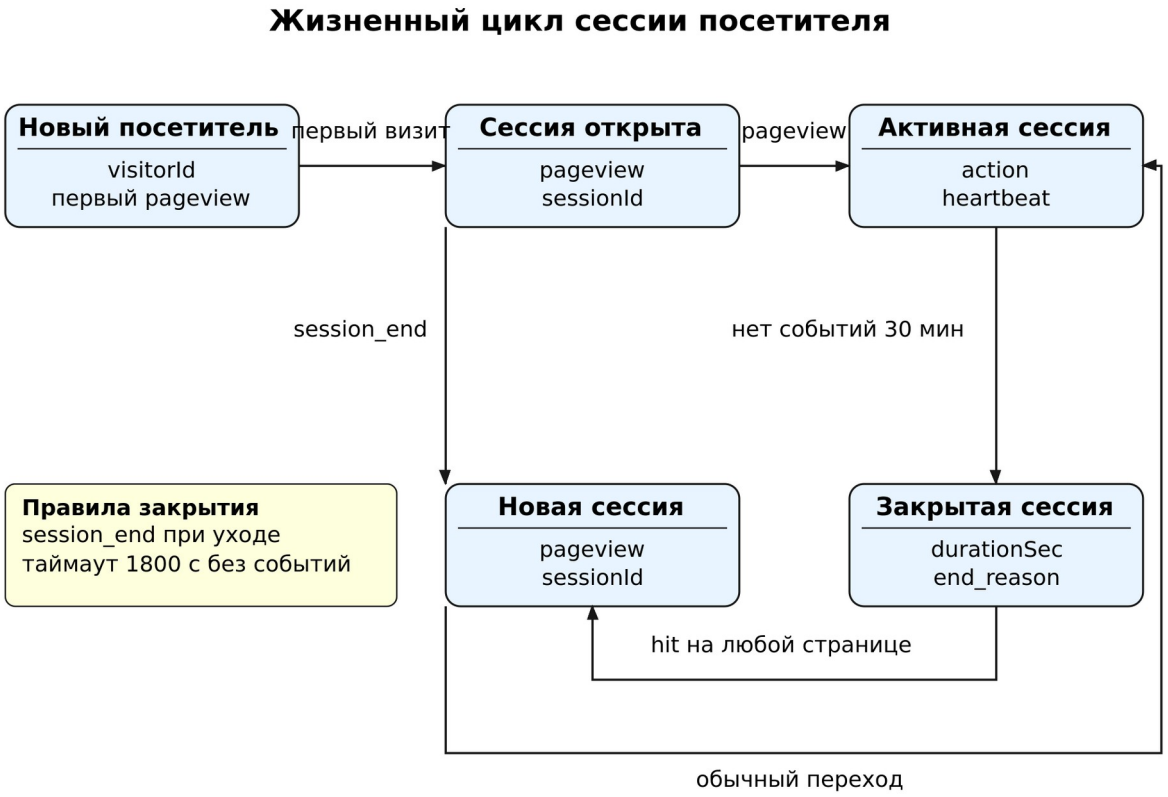


Рисунок 9 — Жизненный цикл сессии посетителя

6. Работа №2. Мобильный клиент под ОС Аврора

6.1 WebSocket-канал сервера

Кроме HTTP-API сервер работы №1 предоставляет канал `ws://<адрес>:3000/ws`. Он реализован вручную по RFC 6455 (`server/websocket.js`: рукопожатие, маскирование кадров, лимит 1 МиБ, закрытие неверного кадра кодом `1002`) и слой подписок и рассылки (`server/realtime.js`: рассылка не чаще, чем раз в 0,5 с после изменения данных). Внешних npm-зависимостей в канале нет.



Рисунок 10 — Обмен сообщениями приложения с сервером по каналу /ws

Таблица 14 — WebSocket-канал сервера

От клиента	Поля	Что делает сервер
`subscribe`	`range` (`24h`, `7d`, `30d`, `all`), `page`, `limit`	запоминает фильтр клиента и присылает снимок статистики
`stats`, `refresh`	—	прислать снимок с текущим фильтром
`pages`	—	прислать список страниц
`track`	`event` (объект как в `POST /api/events`)	записать событие и разослать обновление
`ping`	`t`	проверка связи, ответ `pong`

Таблица 15 — Сообщения сервер → клиент

От сервера	Поля	Когда приходит
`hello`	`service`, `version`, `protocol`, `serverTime`, `heartbeatSec`	сразу после подключения
`stats`	`data` — тот же объект, что отдаёт `GET /api/stats`	после подключения, `subscribe` и изменений данных
`update`	`reason`, `counters`, `serverTime`	при любом изменении данных (лёгкое уведомление)
`event`	`reason`, `event`, `counters`	новое событие (для ленты в реальном времени)
`pages`	`data` — как `GET /api/pages`	в ответ на `pages`
`tracked`	`eventId`, `sessionId`, `counters`	подтверждение записи события из `track`
`pong`	`serverTime`, `echo`	ответ на `ping`
`error`	`error`	некорректное сообщение или тип

Пример реального обмена (проверка канала тестом `test/ws-test.js`):

```
< {"type":"hello","service":"lab01-stat-server","version":"1.0.0",
   "protocol":"aurora-stat/1","heartbeatSec":30}
< {"type":"stats","data":{"totals":{…},"timeline":[…],"devices":{…}}}
> {"type":"subscribe","range":"all"}
< {"type":"stats","data":{"range":{"key":"all"}, …}}
> {"type":"track","event":{"type":"pageview","page":"/aurora"}}
< {"type":"tracked","eventId":"…","sessionId":"…","counters":{…}}
< {"type":"update","reason":"track","counters":{…}}
```

6.2 Клиент StatClient на C++

`StatClient` — единственный класс сетевой части приложения. Он создаётся в `main.cpp` и передаётся в QML как контекстное свойство `StatClient`, поэтому QML-страницы читают статистику как свойства и реагируют на сигналы.

```
connect(m_socket, &QWebSocket::connected, this,
&StatClient::onConnected);
connect(m_socket, &QWebSocket::disconnected, this,
&StatClient::onDisconnected);
connect(m_socket, &QWebSocket::textMessageReceived, this,
&StatClient::onTextMessage);


void StatClient::onConnected()
{
    setConnected(true);
    setStatus(tr("Подключено"));
    m_pingTimer.start();                        // 25 с

    QVariantMap msg;
    msg.insert(QStringLiteral("type"), QStringLiteral("subscribe"));
    msg.insert(QStringLiteral("range"), m_range);
    msg.insert(QStringLiteral("page"), m_pageFilter);
    sendJson(msg);
}

void StatClient::scheduleReconnect()
{
    if (!m_autoReconnect || m_reconnectTimer.isActive())
        return;
    const int delay = qMin(30000, 1000 * (1 << qMin(m_reconnectAttempt,
5)));
    m_reconnectAttempt += 1;
    m_reconnectTimer.start(delay);             // 1, 2, 4, 8, 16, 30 с
}
```

Таблица 16 — Свойства и методы StatClient, доступные из QML

Свойство или метод	**Назначение**
`connected`, `status`, `lastError`	состояние соединения для индикатора на странице
`serverUrl`	адрес канала, читается из `QSettings` (по умолчанию `ws://192.168.1.100:3000/ws`)
`stats`, `sessions`, `pages`, `counters`	последний снимок статистики, таблица сессий, список страниц
`range`, `pageFilter`	фильтр отчёта: период и страница

Свойство или метод	Назначение
`autoReconnect`, `live`, `messageCount`	повторные попытки соединения, приём рассылок, счётчик сообщений
`connectToServer()`, `disconnectFromServer()`, `reconnect()`, `refresh()`	управление соединением и запрос снимка
`requestPages()`	запрос списка страниц
`trackPageView(page, label)`, `trackAction(label, page)`	отправка собственных событий серверу
`formatDuration(sec)`, `formatTime(ms)`	человекочитаемые длительность («1 ч 02 мин») и время

Приложение отправляет события с `ua = "AuroraOS aurorastat/1.0"`, поэтому сервер относит их к устройству `aurora` и они попадают в ту же статистику, что и визиты браузера.

6.3 QML-интерфейс приложения

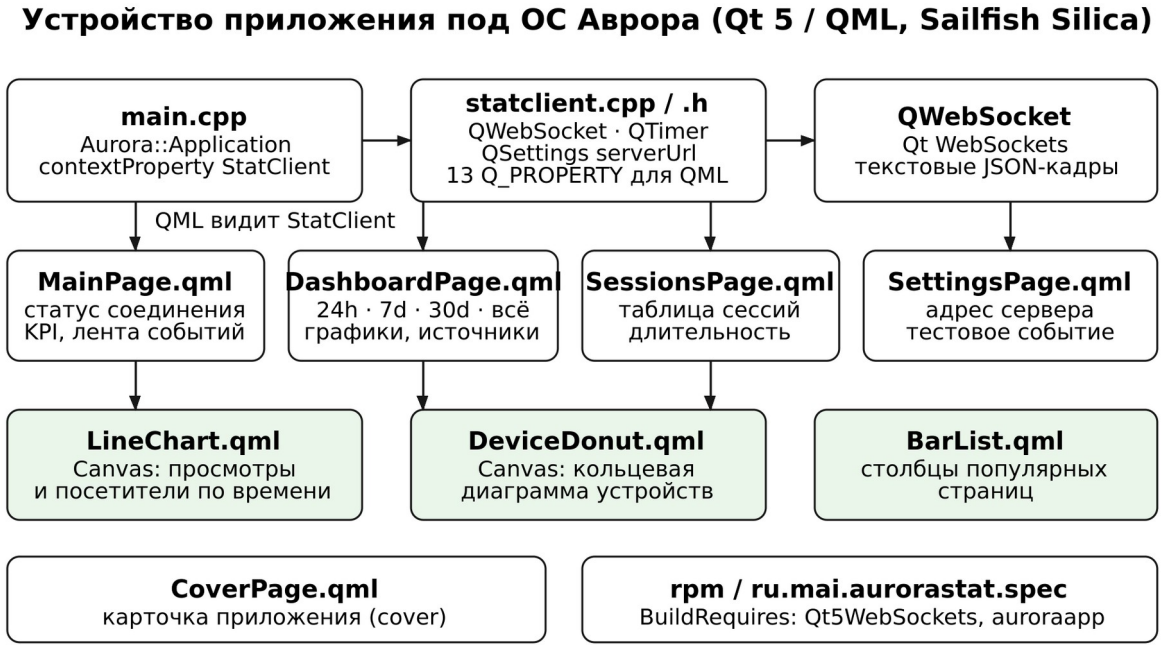


Рисунок 11 — Устройство приложения под ОС Аврора

Таблица 17 — QML-интерфейс приложения

Файл	Что делает	Управление пользователем
`qml/pages/`	главная страница: индикатор	кнопка обновления в

Файл	Что делает	Управление пользователем
`MainPage.qml`	соединения, KPI, лента последних событий	заголовке, кнопки «Открыть дашборд», «Последние сессии», «Проверить сбор: отправить событие», выпадающее меню PullDownMenu
`qml/pages/DashboardPage.qml`	графики: просмотры по времени, устройства, популярные страницы, источники переходов	кнопки периодов `24 ч`, `7 дней`, `30 дней`, `Всё время`
`qml/pages/SessionsPage.qml`	таблица последних сессий: посетитель, длительность, просмотры	прокрутка SilicaFlickable, обновление из PullDownMenu
`qml/pages/SettingsPage.qml`	адрес сервера, автопереподключение, статус, ошибка, счётчик сообщений	`TextField` адреса, кнопки «Применить и подключиться», «Подключиться», «Отключиться», «Обновить данные», «Отправить тестовое событие»
`qml/pages/AboutPage.qml`	описание приложения и протокола обмена	переход из PullDownMenu
`qml/components/LineChart.qml`	`Canvas`: линия просмотров и посетителей по времени	—
`qml/components/DeviceDonut.qml`	`Canvas`: кольцевая диаграмма устройств с легендой	—
`qml/components/BarList.qml`	горизонтальные столбцы популярных страниц	—
`qml/components/StatCard.qml`	карточка KPI	—
`qml/cover/CoverPage.qml`	карточка приложения в переключателе задач	—

6.4 Сборка приложения в Aurora SDK

Сборка делается Aurora SDK 5.1.1.60 (build engine в Docker, цель — snapshot цели сборки):

```
export PATH="$HOME/AuroraOS/bin:$PATH"
cd lab01/aurora-app
sfdk config target=AuroraOS-5.1.1.60-base-x86_64
sfdk build
```

Таблица 18 — Артефакты и параметры сборки

Параметр	Значение
Цель сборки	`AuroraOS-5.1.1.60-base-x86_64` (эмулятор x86_64; для телефона — `aarch64`)
Qt в цели	5.6.3, sysroot цели сборки
Модули Qt	`websockets` (в `.pro`), в пакете — Qt5Core, Qt5Qml, Qt5Quick, Qt5Network, Qt5WebSockets, libauroraapp
Файл сборки пакета	`rpm/ru.mai.aurorastat.spec`, `BuildRequires: pkgconfig(auroraapp)`, `pkgconfig(Qt5WebSockets)`
Результат сборки	`aurora-app/RPMS/ru.mai.aurorastat-1.0.0-1.x86_64.rpm` (72 КБ)
Зависимости пакета	`sailfishsilica-qt5 >= 0.10.9`, `qt5-qtwebsockets`
Локализация	`translations/ru.mai.aurorastat.ts`, …`-ru.ts` (`lupdate`, `lrelease` при сборке)

Модуль `QtWebSockets` в SDK берётся из репозитория `omp`: без `pkgconfig(Qt5WebSockets)` в `spec` пакет не собирается, это единственная правка сборки, которая понадобилась.

6.5 Установка и запуск на эмуляторе Aurora OS

Устройство для демонстрации — эмулятор Aurora OS 5.1.1.60 (VirtualBox, модель Mashtab TrustPhone T1, 720×1600). Сервер при этом запущен на ПК, то есть есть и «удалённое устройство», и подключение по локальной сети.

Таблица 19 — Установка и запуск на эмуляторе Aurora OS

Шаг	Команда	Результат
Поднять эмулятор	`VBoxManage startvm "AuroraOS-5.1.1.60-base" --`	Aurora 5.1.1.60, ssh порт 2223

Шаг	Команда	Результат
	`type headless`	
Положить пакет	`scp …/RPMS/*.rpm` → `/tmp` на устройстве	файл в `/tmp` устройства
Установить	`sudo pkcon install-local -y /tmp/*.rpm`	`Installed ru.mai.aurorastat`
Указать адрес сервера	`~/.config/ru.mai/aurorastat.conf`: `serverUrl=ws://10.0.2.2:3000/ws`	приложение подключается к ПК сразу при старте
Запустить	`env XDG_RUNTIME_DIR=/run/user/100000 WAYLAND_DISPLAY=../../display/wayland-0 QT_QPA_PLATFORM=wayland /usr/bin/ru.mai.aurorastat`	процесс `/usr/bin/ru.mai.aurorastat` в сессии Wayland
Проверить канал с устройства	`curl` с заголовками `Upgrade: websocket` к `http://10.0.2.2:3000/ws`	`HTTP/1.1 101 Switching Protocols`, затем `hello` и `stats` от сервера
Проверить на стороне сервера	`ss -tn state established '( sport = :3000 )'`	соединение от эмулятора держится более 5 минут (кадры `ping` каждые 30 с)

Адрес `10.0.2.2` — адрес хоста из точки зрения гостя VirtualBox (NAT). Сервер слушает `0.0.0.0:3000`, поэтому доступнен и по адресу ПК в Wi-Fi сети (`http://192.168.1.163:3000/`), и по адресу, который печатает сам сервер при запуске.

7. Архитектура обеих работ

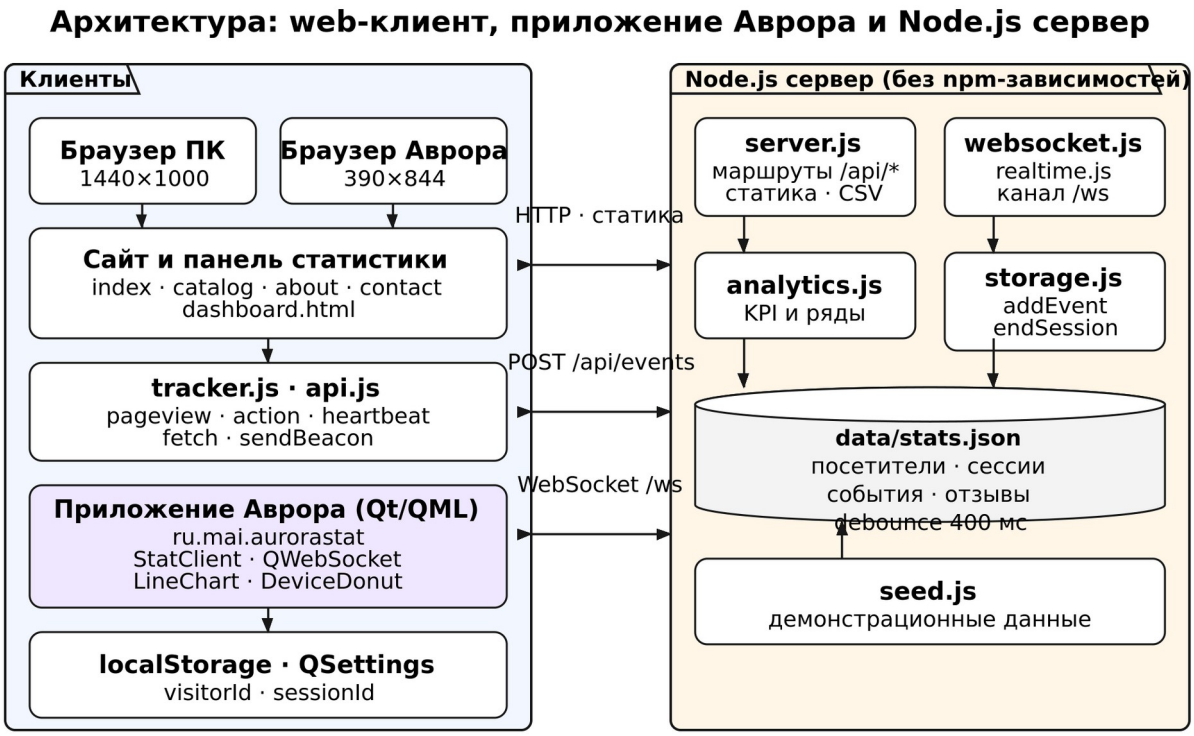


Рисунок 12 — Архитектура: web-клиент, приложение Аврора и Node.js сервер

Таблица 20 — Архитектура обеих работ

Модуль	Ответственность
`server/server.js`	HTTP, маршруты `/api/*`, валидация тела, раздача статики, CSV, печать адресов локальной сети
`server/websocket.js`	рукопожатие и кадры WebSocket, лимит размера сообщения, корректное закрытие
`server/realtime.js`	подписки клиентов, рассылка `stats`, `update`, `event`, подтверждение `tracked`
`server/storage.js`	загрузка и сохранение JSON, посетители, сессии, сортировка событий, лимиты
`server/analytics.js`	агрегаты: KPI, таймлайн, топ страниц, устройства, длительности, источники
`server/seed.js`	детерминированный генератор демонстрационных посетителей за N дней
`public/js/tracker.js`	асинхронный сбор статистики с клиента
`public/js/api.js`	`async`-обёртка `fetch` + `AbortController` (таймаут 8 с)
`public/js/charts.js`	рисование графиков на canvas

Модуль	Ответственность
`public/js/dashboard.js`	форма отчёта, KPI, графики, таблицы, автообновление
`public/js/app.js`	меню, счётчики, фильтр каталога, форма отзывов
`aurora-app/src/statclient.cpp`	WebSocket-клиент приложения: подписка, приём, отправка событий, переподключение
`aurora-app/qml/`	страницы и графики интерфейса Авроры

8. Тестирование

Таблица 21 — Тестирование

Проверка	Команда	Результат
API сервера: health, события, посетители, сессии, длительность, отказ, глубина, источники, CSV, очистка	`npm test`	33 проверки из 33 пройдено
WebSocket-канал: рукопожатие, `hello`, `subscribe`, `stats`, `track` + `tracked`, `update`, `pages`, `ping`/`pong`, неверный кадр → код 1002	`npm run test:ws`	9 проверок из 9 пройдено
Smoke-тест интерфейса (jsdom): загрузка страниц, наличие кнопок и форм, обработчики, корректность JS	`npm run test:ui`	16 проверок из 16 пройдено
Сборка приложения под Аврору	`sfdk build`	RPM собран, зависимости пакета разрешены
Установка и запуск на устройстве	`pkcon install-local`, запуск бинаря в сессии Wayland	приложение работает на эмуляторе
Канал с мобильного устройства	`curl` с upgrade-заголовками из гостя + `ss -tn` на сервере	`101 Switching Protocols`, соединение держится

Тесты API и WebSocket не требуют npm-пакетов: это обычные Node.js-скрипты (`test/api-test.js`, `test/ws-test.js`), они поднимают

сервер на временном порту и с временным файлом статистики.

9. Демонстрация работы

9.1 Запуск в локальной сети

```
cd lab01/stat-server
node server/server.js            # или npm start



Сервер статистики запущен:
  http://localhost:3000/
  http://192.168.1.163:3000/     ← адрес для телефона в той же Wi-Fi сети
Панель статистики: <адрес>/dashboard.html
WebSocket-канал: ws://<адрес>:3000/ws
```

Демонстрационные данные (посетители за 7 дней) добавляются командой `npm run seed` или кнопкой «Демо-данные» на панели статистики.

9.2 Сводка за демонстрационный прогон

Таблица 22 — Сводка за демонстрационный прогон

Показатель	**Значение**
Просмотры страниц	138
Уникальные посетители	25
Сессии	54
Новые посетители	14
Вернувшиеся посетители	11
Нажатия кнопок форм (`action`)	39
Средняя длительность сессии	221 с
Максимальная длительность сессии	588 с
Глубина просмотра	2,56 страницы на сессию
Показатель отказов	44,4 %
Активные сейчас	5

Показатель	Значение
Устройства	desktop 34, aurora 12, mobile 6, tablet 2
Источники	прямой заход 27, внутренние переходы 11, внешние: поиск, соцсеть, рассылка

9.3 Скриншоты: браузер десктопной ОС

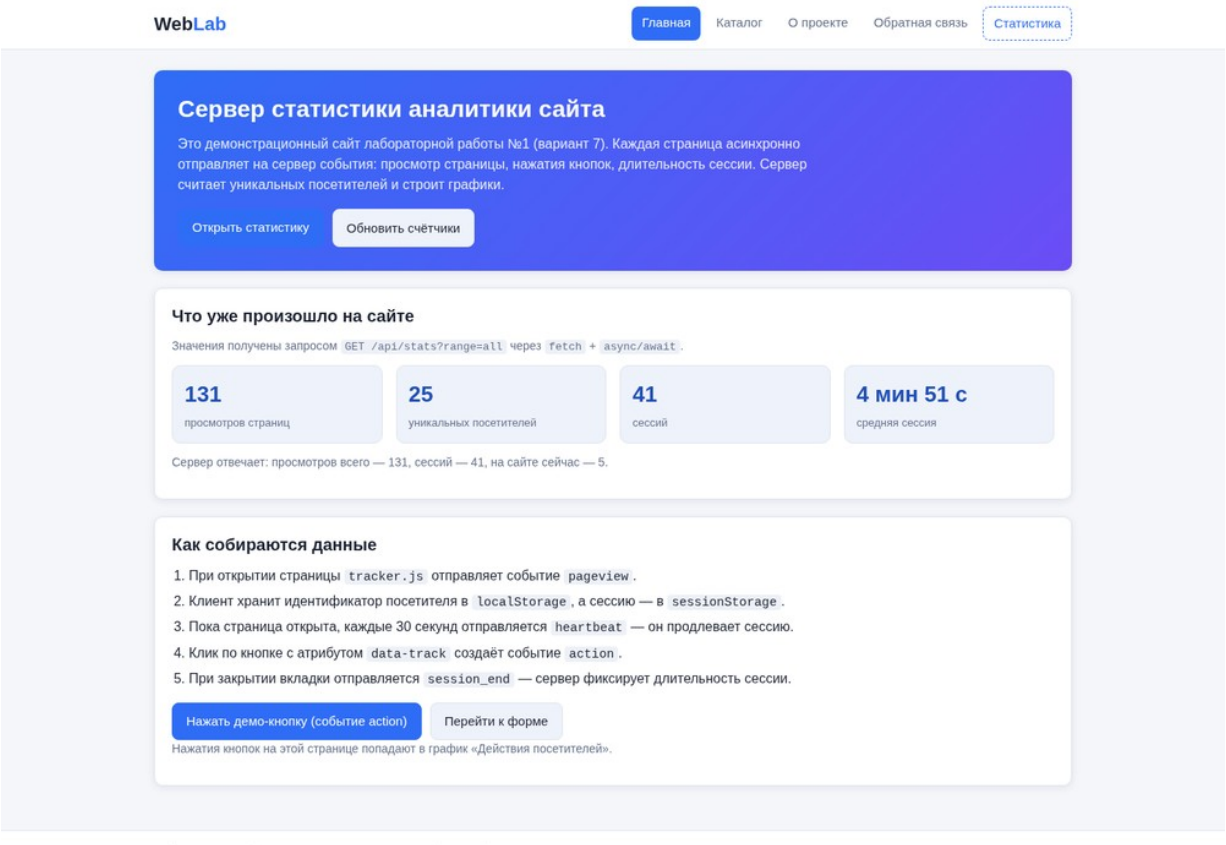


Рисунок 13 — Главная страница в браузере ПК

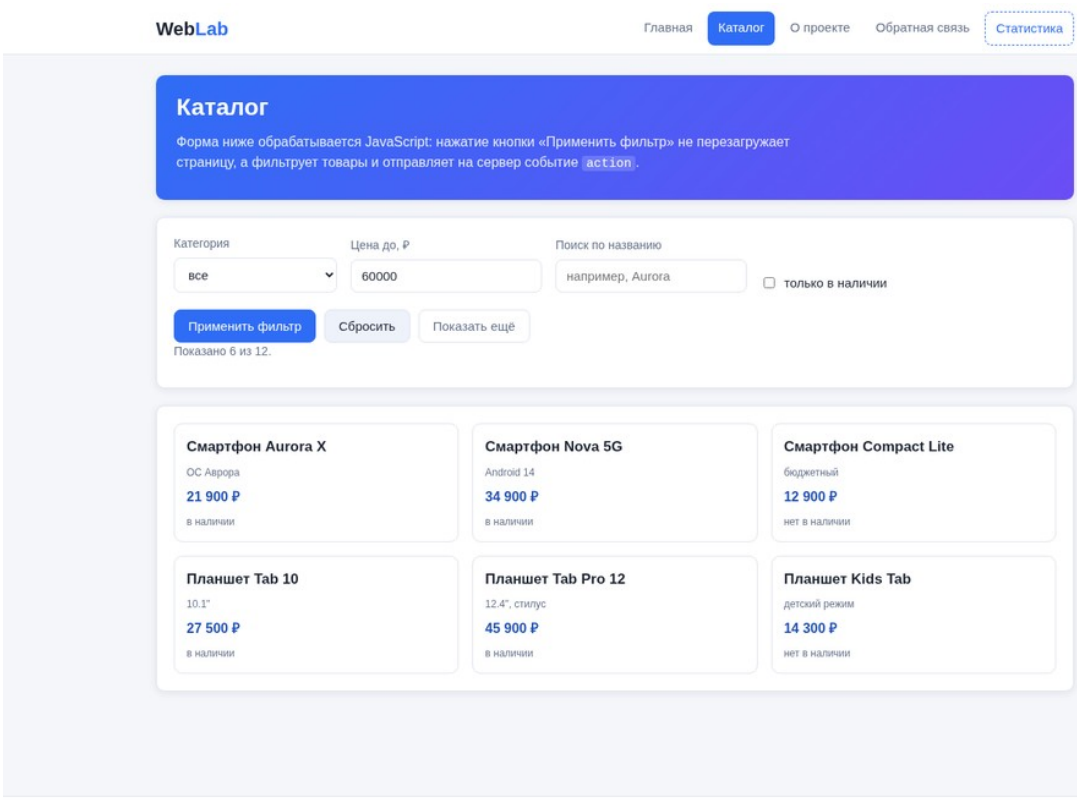


Рисунок 14 — Каталог с формой фильтра

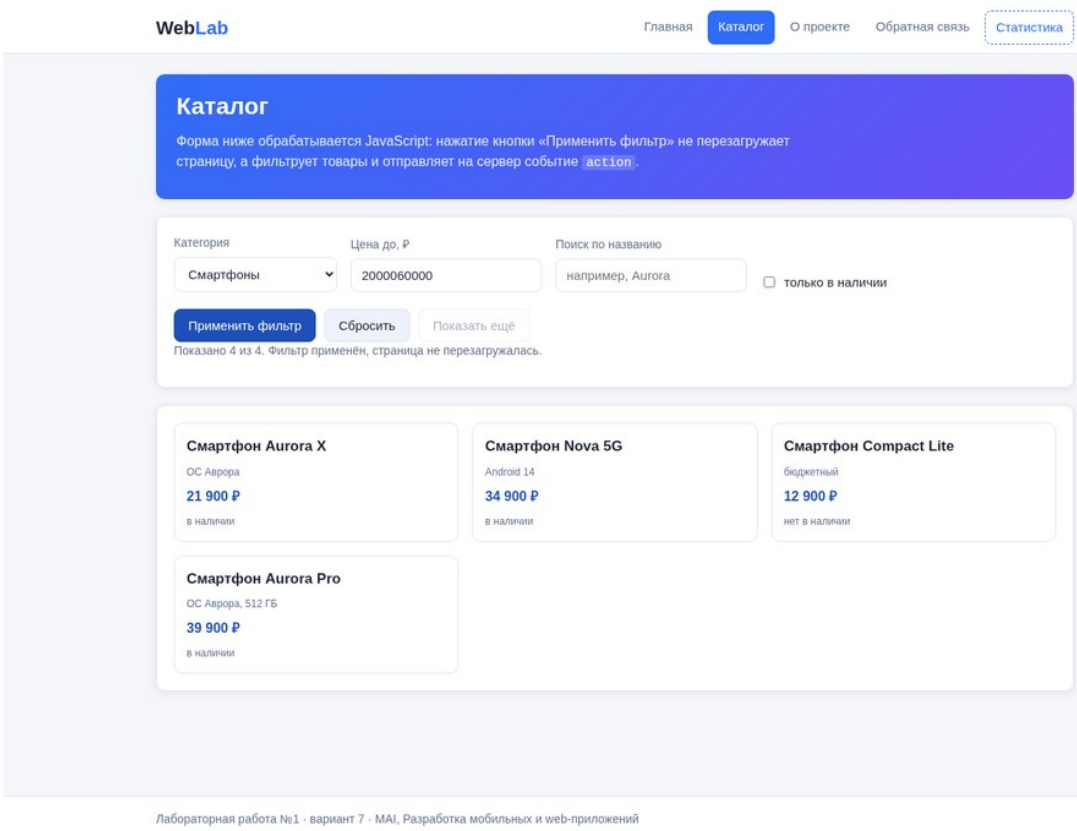


Рисунок 15 — Каталог после нажатия кнопки «Применить» в форме фильтра

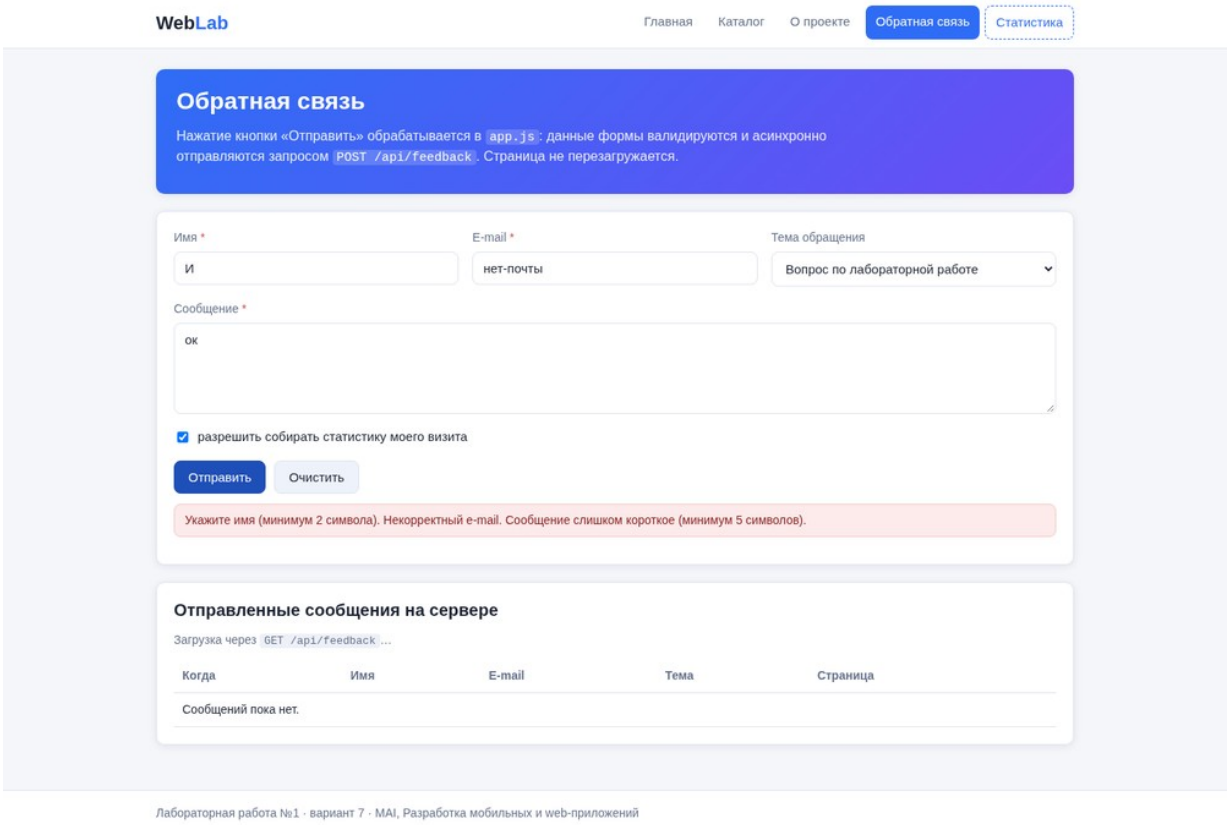


Рисунок 16 — Форма обратной связи: проверка полей после отправки

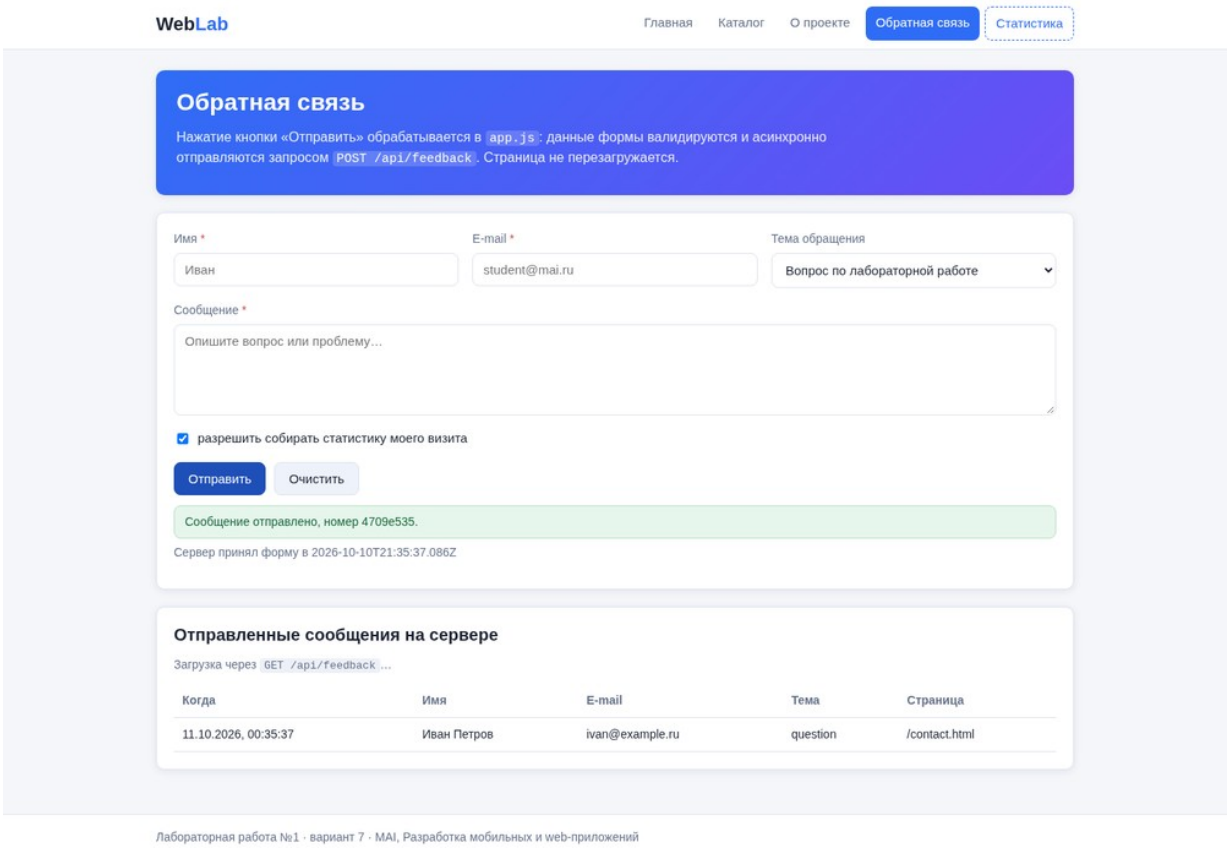


Рисунок 17 — Форма обратной связи: сообщение сохранено и добавлено в таблицу

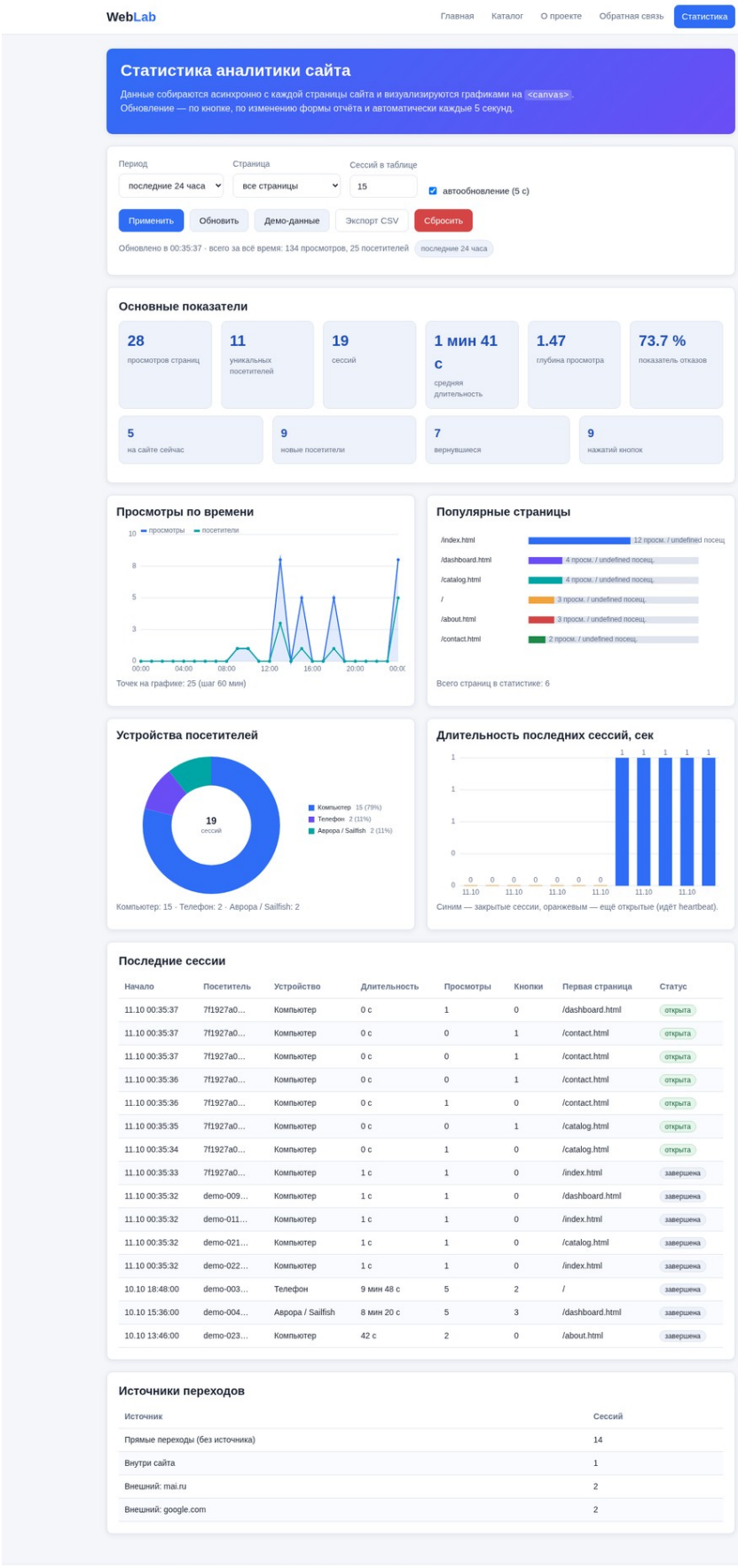


Рисунок 18 — Панель статистики на ПК

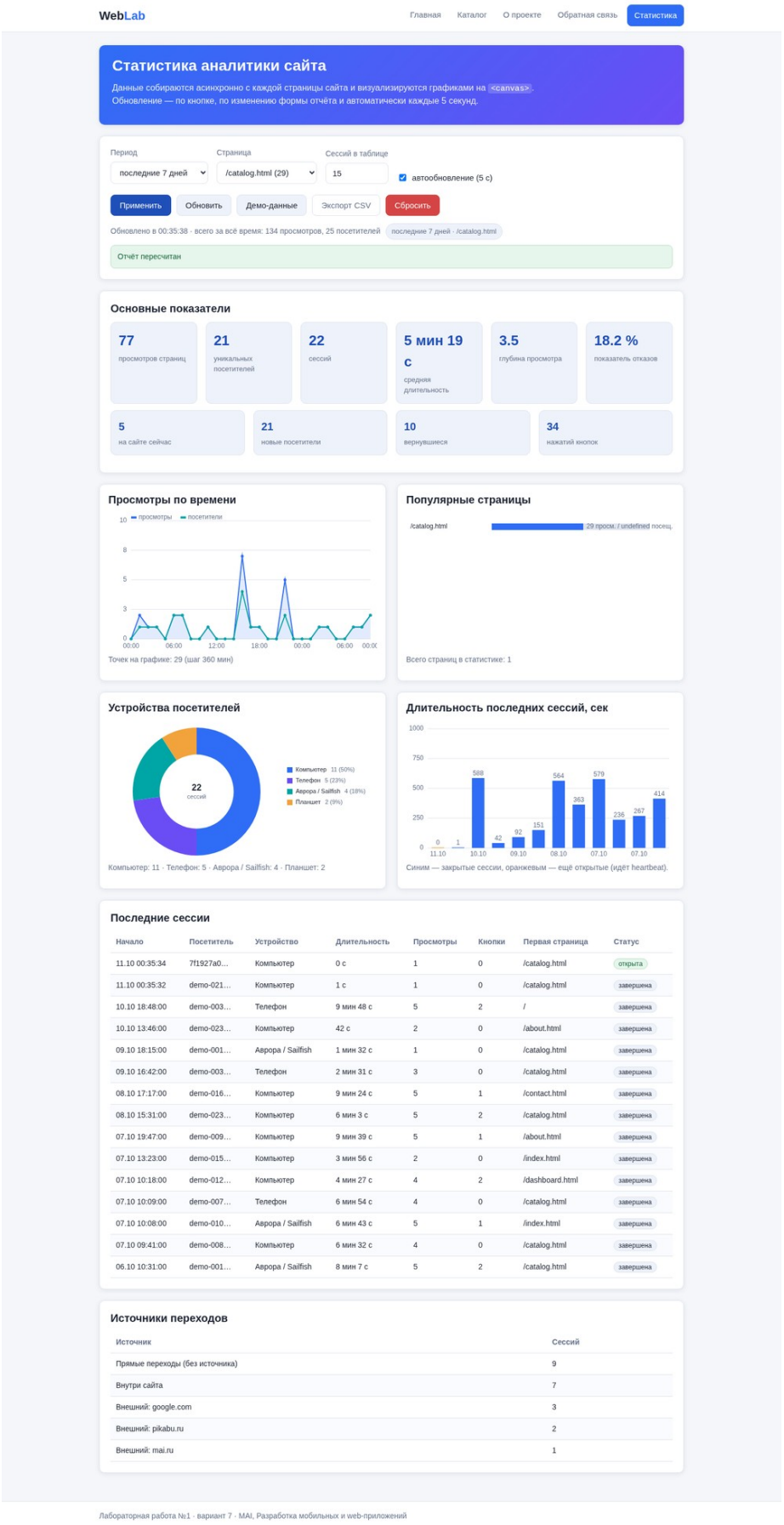


Рисунок 19 — Панель статистики: форма отчёта с фильтром «7 дней» и страница «Каталог»

9.4 Скриншоты: мобильный браузер (ОС Аврора)

Снимки сделаны браузером устройства под управлением Аврора ОС (User-Agent `Mozilla/5.0 (Aurora OS 3.0; Phone) AppleWebKit/537.36` …), сервер доступен по адресу ПК в локальной сети.

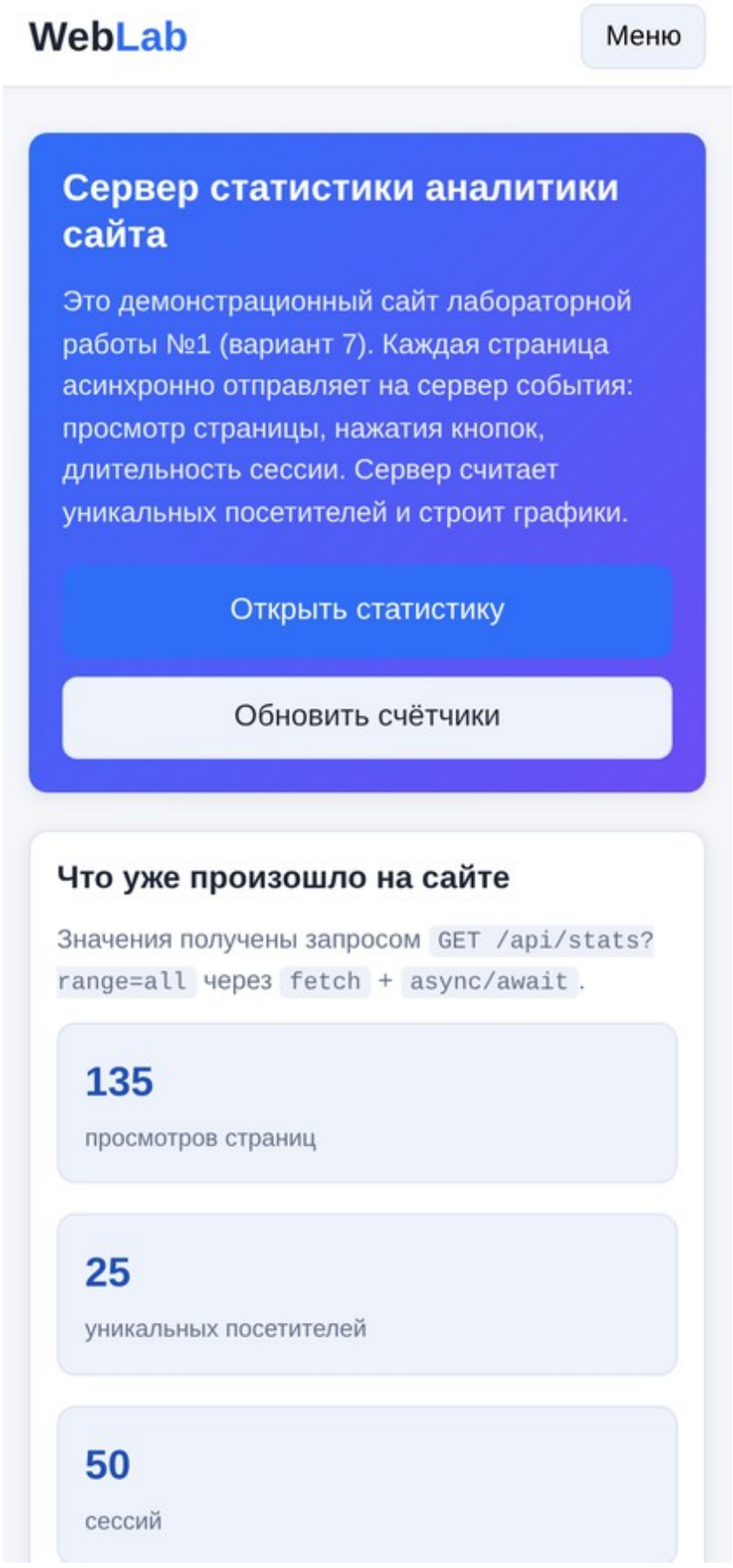


Рисунок 20 — Главная страница в мобильном браузере Аврора

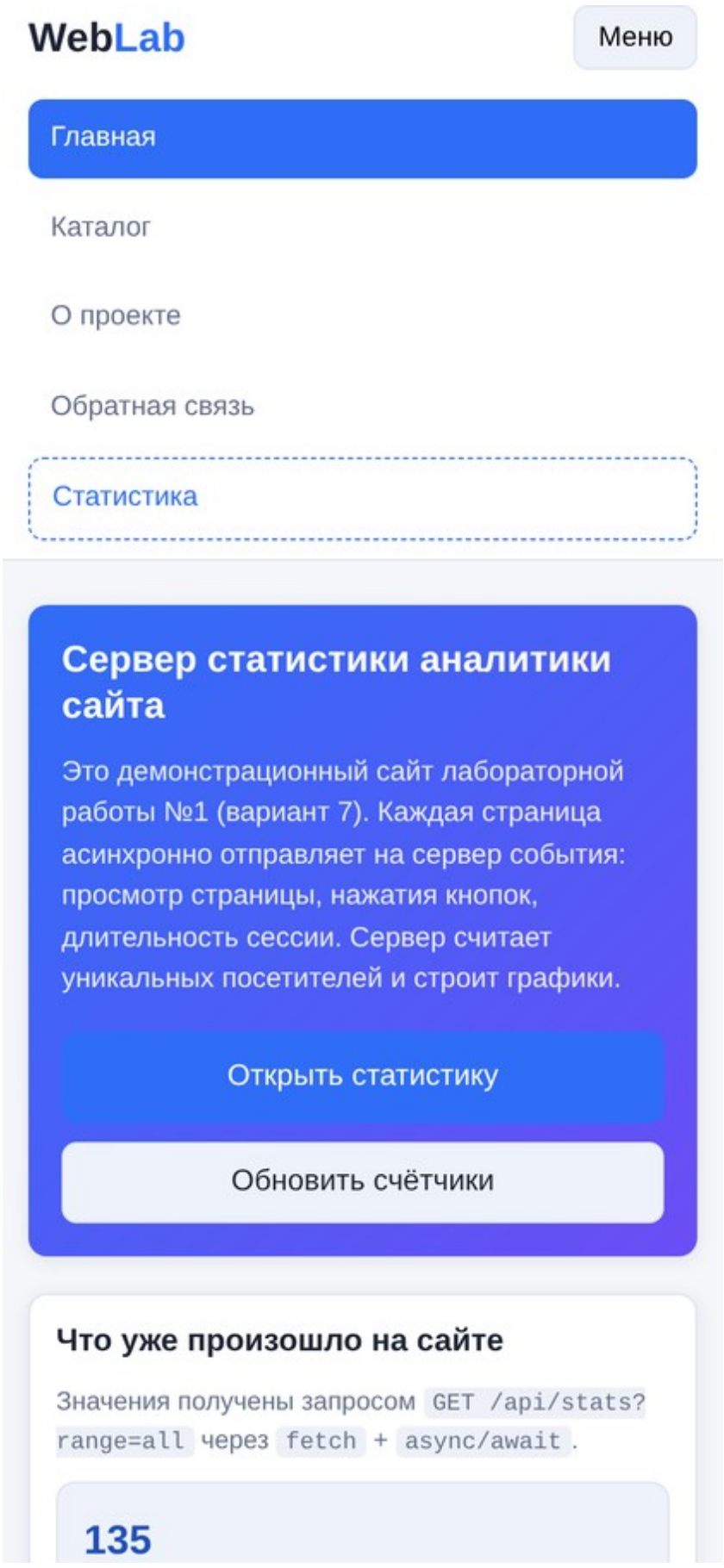


Рисунок 21 — Меню, открытое кнопкой при адаптивной вёрстке

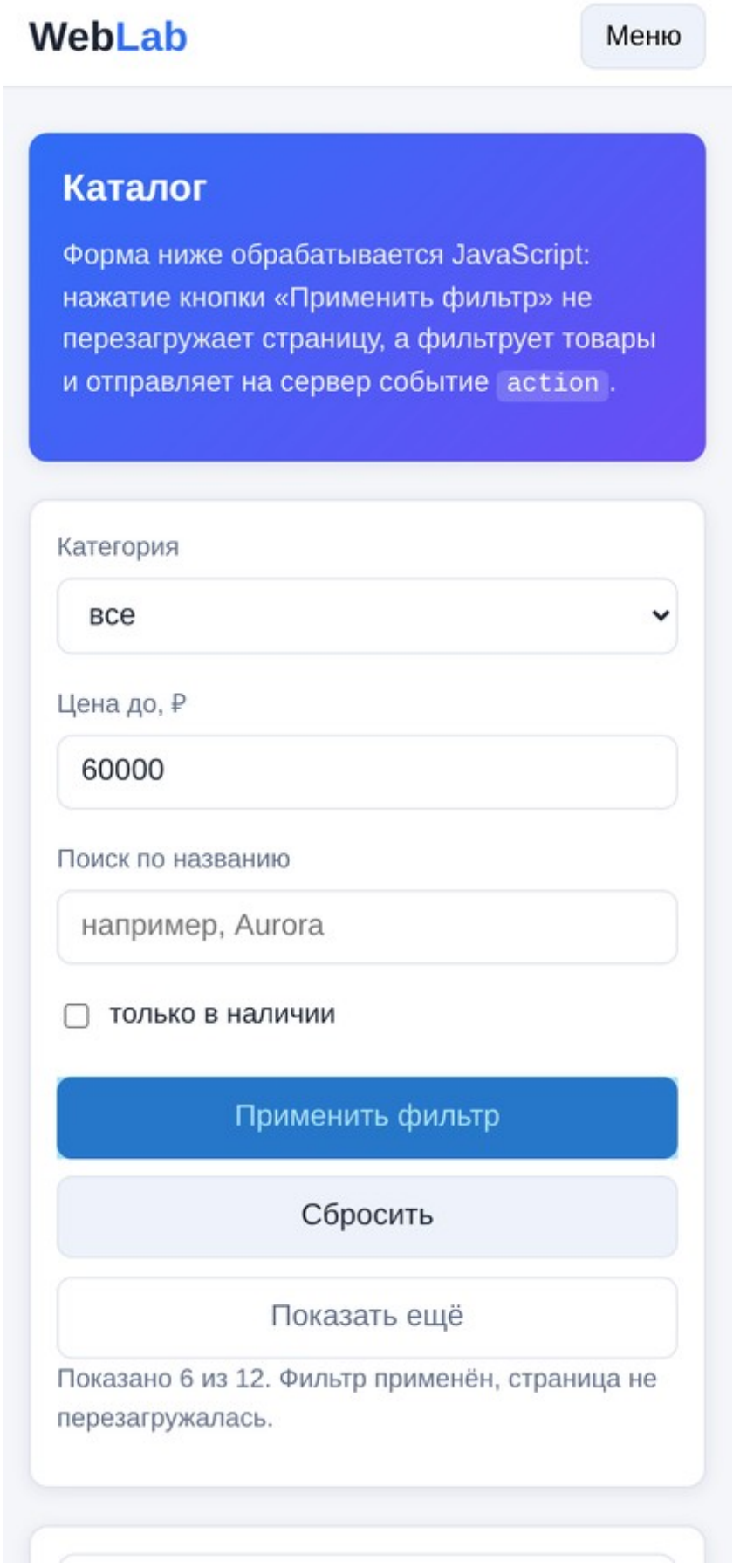


Рисунок 22 — Каталог на телефоне: формы на всю ширину

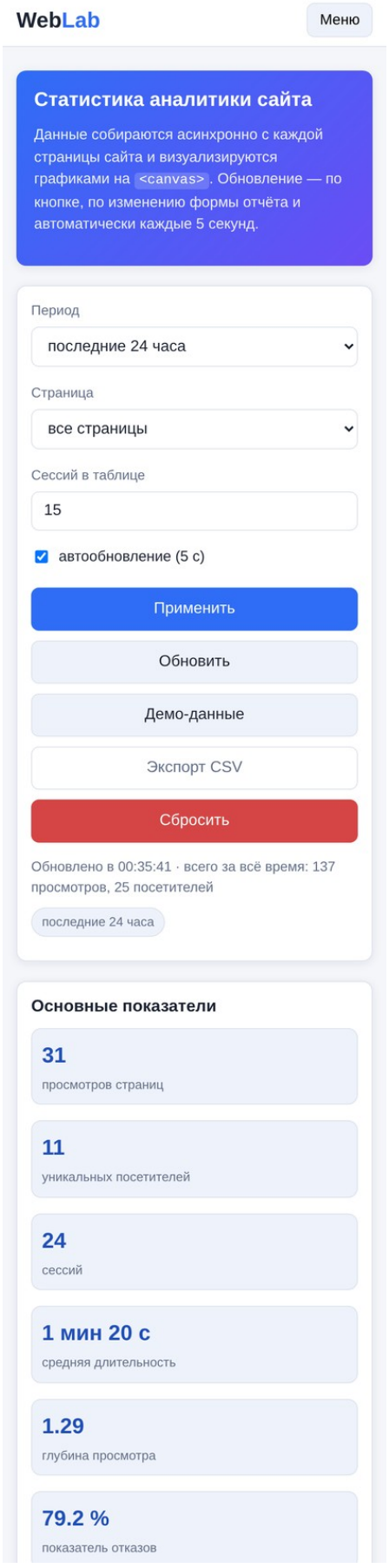


Рисунок 23 — Панель статистики на телефоне: KPI и график по времени

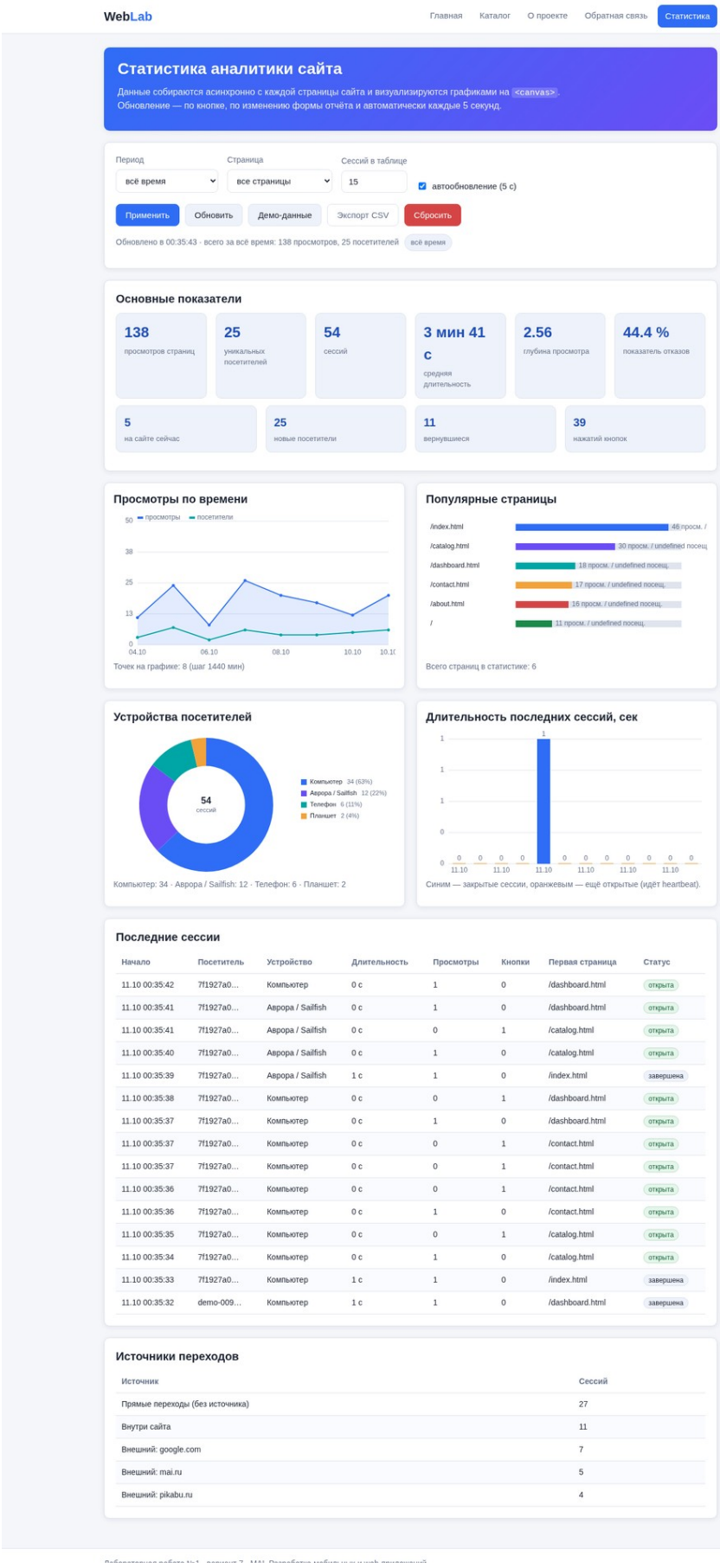


Рисунок 24 — Панель статистики на телефоне: диаграмма устройств

9.5 Скриншот приложения на эмуляторе Аврора

Приложение `ru.mai.aurorastat` (работа №2) запущено на эмуляторе Aurora OS 5.1.1.60 и подключено к серверу на ПК по каналу `ws://10.0.2.2:3000/ws`.

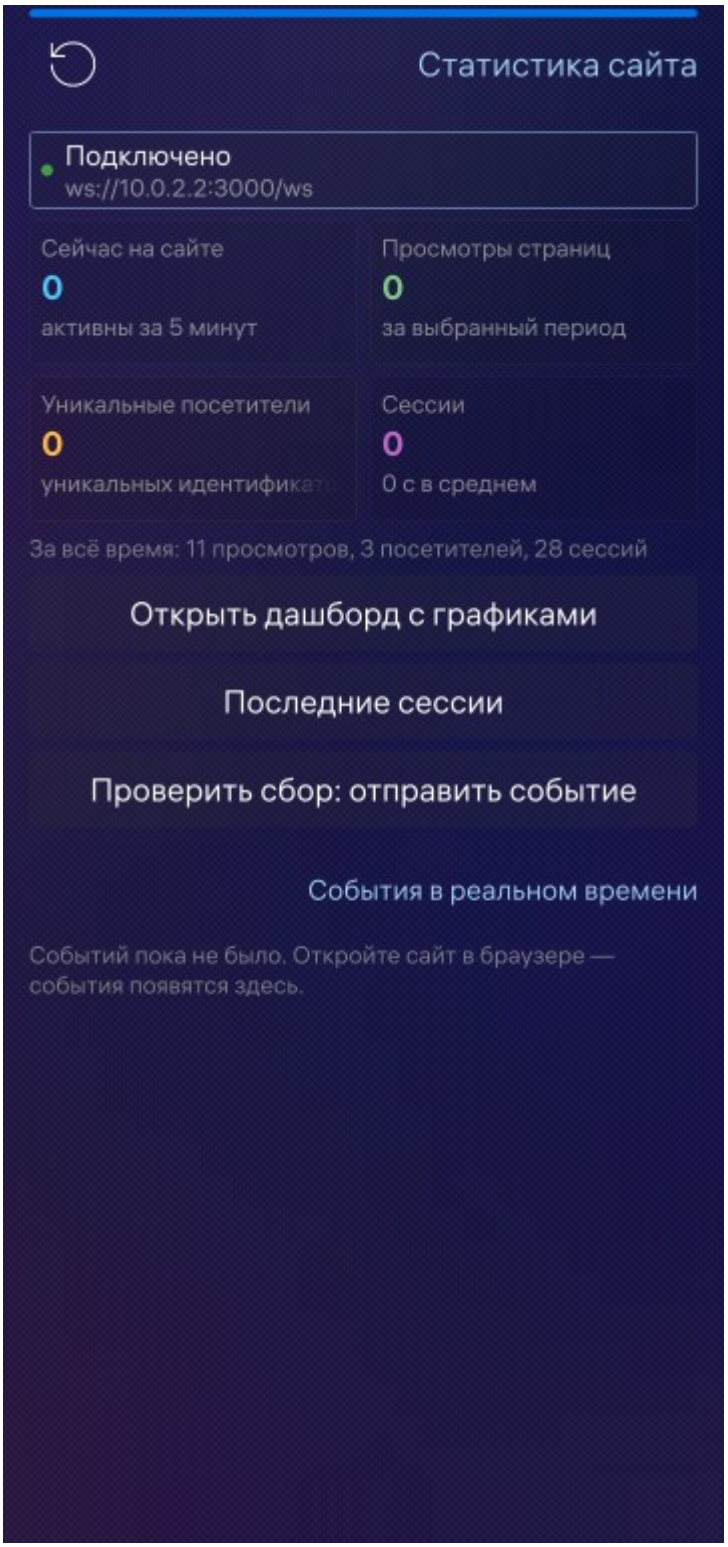


Рисунок 25 — Приложение статистики на эмуляторе Aurora OS 5.1.1.60

10. Проверка требований варианта №7

Таблица 23 — Проверка требований варианта №7

Требование варианта	Реализация	Где проверить
Сервер статистики аналитики сайта	Node.js-сервер с API и хранилищем	`server/server.js`, `server/storage.js`
Количество просмотров страниц	`totals.pageViews`, топ страниц, таймлайн	`GET /api/stats`, `analytics.js`
Уникальные посетители	`visitorId` + `visitors` в хранилище, новые и вернувшиеся	`tracker.js`, `storage.js`
Длительность сессий	`durationSec` у сессии, средняя длительность, гистограмма	`storage.js`, `analytics.js`
Сбор данных асинхронно через JS	`async/await`, `fetch`, `sendBeacon`, очередь событий, heartbeat	`public/js/tracker.js`
Сохранение на сервере	`data/stats.json`, асинхронная запись с debounce	`server/storage.js`
Визуализация графиками или диаграммами	4 canvas-графика на дашборде, 4 в приложении Авроры, 6 в отчёте	`charts.js`, `qml/components/`, `report/images/`
Web-интерфейс с обработкой нажатий кнопок	5 страниц, 3 формы, 8 кнопок панели	`app.js`, `dashboard.js`
Асинхронный вызов JS	`async`, `promise`, `Promise.all`, `AbortController`	`api.js`, `dashboard.js`
Адаптивная вёрстка `@media`	4 брейкпоинта + тёмная тема + печать	`public/css/`
Запуск в локальной сети	`0.0.0.0:3000`, вывод адресов машины	вывод сервера
Демонстрация в браузере ПК и Аврора	снимки 9.3 и 9.4	раздел 9
Мобильный клиент на Qt с web-сокетами	`StatClient` + QML-интерфейс, RPM для Авроры	`lab01/aurora-app/`

11. Выводы

1.Работа №1 выполнена полностью: сервер на Node.js без внешних

зависимостей, 5 страниц web-интерфейса, асинхронный сбор статистики через JS, сохранение в JSON и визуализация графиками. Все требования задания проверены: `async`/`promise` в клиентском коде, адаптивная вёрстка через `@media`, обработка нажатий кнопок форм, запуск в локальной сети, демонстрация в браузере ПК и в браузере мобильного устройства под управлением Аврора. Smoke-тест интерфейса подтверждает это программно: 16 проверок из 16 (формы, кнопки, графики на canvas, автообновление панели).

2. Метрики варианта №7 считаются по реальным событиям: просмотры страниц, уникальные посетители (с разделением на новых и вернувшихся) и длительность сессий с закрытием по таймауту или по событию `session_end`.

3. При проверке демо-данных найдены и исправлены 4 ошибки учёта (дубли идентификаторов сессий, несохранение порядка событий, неверное определение источника перехода); после исправления тесты API проходят полностью — 33 проверки из 33.

4. Работа №2 выполнена на Qt/QML (Aurora SDK 5.1.1.60): приложение `ru.mai.aurorastat` собирается в пакет RPM, устанавливается на устройство под управлением Авроры и обменивается с сервером работы №1 данными через web-socket-канал `/ws`, реализованный на сервере вручную по RFC 6455 (9 проверок канала пройдено).

5. Связь «приложение ↔ сервер» подтверждена на эмуляторе Aurora OS 5.1.1.60: рукопожатие `101 Switching Protocols`, сервер отвечает `hello` и `stats`, соединение удерживается за счёт `ping`/`pong` каждые 25 с на клиенте и heartbeat 30 с на сервере.

6. Обе работы дают один проект: web-клиент (браузер) и мобильный клиент (Qt/QML) используют одно хранилище и одни метрики, поэтому статистика с телефона и с ПК суммируется в тех же графиках.

12. Как воспроизвести материалы отчёта

Таблица 24 — Как воспроизвести материалы отчёта

Действие	Команда
Запустить сервер	`cd lab01/stat-server && npm start`
Добавить демонстрационные данные	`cd lab01/stat-server && npm run seed`
Проверить API	`cd lab01/stat-server && npm test`
Проверить WebSocket-канал	`cd lab01/stat-server && npm run test:ws`
Проверка интерфейса	`cd lab01/stat-server && npm install jsdom && npm run test:ui`
Построить графики отчёта	`python3 lab01/stat-server/report/src/python/make_charts.py`
Построить схемы отчёта	`python3 lab01/report/src/diagrams.py`
Собрать DOCX и PDF отчёта	`python3 ~/.agents/skills/gost-docx-report/scripts/build_report.py -c lab01/report/report.yaml --check`
Собрать приложение под Аврору	`cd lab01/aurora-app && sfdk config target=AuroraOS-5.1.1.60-base-x86_64 && sfdk build`
Поднять эмулятор Авроры	`VBoxManage startvm "AuroraOS-5.1.1.60-base" --type headless`
Установить приложение на устройство	`scp …/RPMS/*.rpm defaultuser@127.0.0.1:/tmp/ && ssh -p 2223 defaultuser@127.0.0.1 sudo pkcon install-local -y /tmp/…rpm`

Таблица 25 — Содержание каталога документации lab01/stat-server/docs/

Файл	Содержание
`01-requirements.md`	требования задания и варианта
`02-architecture.md`	архитектура сервера и клиента
`03-api.md`	HTTP API
`04-data-model.md`	модель данных
`05-frontend.md`	web-интерфейс и JS
`06-testing.md`	тесты
`07-run-manual.md`	руководство по запуску
`08-user-manual.md`	руководство пользователя

Файл	Содержание
`09-aurora-browser.md`	работа в браузере Аврора
`10-websocket.md`	протокол web-socket-канала

13. Полезные команды

Таблица 26 — Полезные команды

Задача	Команда
Посмотреть статистику без браузера	`curl -s localhost:3000/api/stats?range=7d | python3 -m json.tool`
Выгрузить CSV	`curl -s "localhost:3000/api/export.csv?type=sessions" -o sessions.csv`
Проверить, кто подключён по WebSocket	`ss -tn state established '( sport = :3000 )'`
Подключиться к эмулятору по ssh	`ssh -p 2223 -i ~/AuroraOS/vmshare/ssh/private_keys/sdk defaultuser@127.0.0.1`
Запустить приложение на устройстве	`env XDG_RUNTIME_DIR=/run/user/100000 WAYLAND_DISPLAY=../../display/wayland-0 QT_QPA_PLATFORM=wayland /usr/bin/ru.mai.aurorastat`
Проверить версию ОС на устройстве	`ssh -p 2223 … cat /etc/os-release`
Список целей Aurora SDK	`sfdk config --show target`